P.Vishwa nandan
192473015

TASK 8 – Apply Polymorphism

Marks: 4

This task is about **Polymorphism in Java**. Polymorphism means one method or reference can work with different types of objects.

The assignment requires:

1. Compile-time polymorphism (method overloading).
2. Runtime polymorphism (method overriding).
3. Upcasting using a Vehicle array.
4. Dynamic method dispatch.
5. A completed Java program using inheritance, overriding and an interface.
6. Alternative concept and complexity analysis. CO1 JAVA

1. Compile-Time Polymorphism (Method Overloading)

Method overloading means creating multiple methods with the same name but different parameters.

In our parking system, we can calculate parking fees in two ways.

Example:

double calculateFee(int minutes)

This calculates the fee using parking duration in minutes.

Another method:

double calculateFee(LocalTime in, LocalTime out)

This calculates the fee using entry and exit times.

Example implementation:

```
import java.time.LocalTime;

import java.time.Duration;


class ParkingFee {


   double calculateFee(int minutes) {
```

```
        return (minutes + 59) / 60.0;
    }

    double calculateFee(LocalTime in, LocalTime out) {
        long minutes = Duration.between(in, out).toMinutes();

        if (minutes < 0) {
            minutes += 1440;
        }

        return (minutes + 59) / 60.0;
    }
}
```

Explanation:

Both methods have the same name but different parameters.

The compiler selects the appropriate method based on the arguments passed.

This is called compile-time polymorphism.

2. Runtime Polymorphism (Method Overriding)

Runtime polymorphism occurs when a subclass provides its own implementation of a parent class method.

In our parking system, different vehicles have different parking rates.

For example:

- TwoWheeler: Rs. 10 per hour.
- Car: Rs. 30 per hour.
- ElectricCar: Rs. 30 per hour.

The ratePerHour() method is overridden in the vehicle subclasses.

Java selects the appropriate implementation based on the actual object created.

This is called runtime polymorphism.

3. Upcasting Using a Vehicle Array

Upcasting means storing a child-class object in a parent-class reference.

For example:

Vehicle v1 = new TwoWheeler("TN01A1");

Vehicle v2 = new Car("TN02B2");

Vehicle v3 = new ElectricCar("TN03C3");

We can also store different vehicle objects in a single array.

Example:

```
Vehicle[] vehicles = {
   new TwoWheeler("TN01A1"),
   new Car("TN02B2"),
   new ElectricCar("TN03C3")
};
```

Here, the Vehicle array stores objects of three different vehicle types.

This makes it easier to manage different vehicles using a common parent-class reference.

4. Dynamic Method Dispatch

Dynamic method dispatch means Java selects the appropriate overridden method at runtime based on the actual object.

Example:

Vehicle v = new Car("TN02B2");

double fee = v.calculateFee(130);

Here:

- Declared reference type: Vehicle.
- Actual object type: Car.
- Called method: calculateFee(130).

The calculateFee() method in Vehicle calls ratePerHour().

Since the actual object is Car, Java executes the overridden ratePerHour() method in Car, which returns Rs. 30.

Therefore, the parking fee is Rs. 90.

This avoids writing long if-else conditions for different vehicle types.

5. Why Polymorphism Is Useful

Without polymorphism, we may need to use multiple if-else conditions to check whether a vehicle is a car, two-wheeler or electric vehicle.

With polymorphism:

1. All vehicles can be stored using Vehicle references.
2. Each vehicle provides its own implementation.
3. Java automatically selects the appropriate method.
4. Code becomes easier to extend and maintain.